TASK 1 – UNDERSTAND THE PROBLEM AND IDENTIFY THE OBJECTS

1.1 Problem Analysis

The Namma Metro Smart-Card Fare System calculates passenger fares according to the farthest zone travelled. The network consists of three zones: Inner, Middle and Outer.

The base fares are:

Zone	**Base Fare**
Inner	Rs. 10
Middle	Rs. 20
Outer	Rs. 30

A 20% surcharge is applied during peak hours (8:00–10:00 AM and 5:00–8:00 PM). Passengers use stored-value smart cards to pay for journeys. A journey is permitted only when the remaining card balance after fare deduction is at least Rs. 50. Students and senior citizens are eligible for concessions.

The system must calculate fares accurately, prevent negative balances, apply the correct peak-hour rules and maintain the fare information consistently.

1.2 Identification of Objects, Data and Behaviour

An object represents an entity that combines data (attributes) and behaviour (functions).

Object	**Data (Attributes)**	**Behaviour (Methods)**	**Justification**

SmartCard	Card ID, balance	checkBalance(), canTravel(), deductFare(), recharge()	Maintains the passenger's stored value and ensures the minimum balance rule is followed.
Passenger	Passenger ID, passenger category	getCategory(), getConcession()	Identifies whether the passenger is a regular passenger, student or senior citizen so that the applicable concession can be considered.
Journey	Source zone, destination zone, journey time, calculated fare	calculateFare(), checkPeakHour(), completeJourney()	Represents an individual journey and handles its fare calculation and journey details.
Zone	Zone name, zone number	getZoneName(), getZoneNumber()	Represents the metro zones and helps identify the farthest zone travelled.
FareTable	Base fares, surcharge percentage, minimum balance	getBaseFare(), getSurcharge(), getMinimumBalance()	Stores fare rules centrally so that fare changes can be managed consistently.

These objects are related because a passenger uses a smart card to pay for a journey. The journey uses the zone and fare table information to calculate the applicable fare. The smart card then checks whether the journey is affordable under the minimum balance condition.

1.3 Procedural Approach vs Object-Oriented Approach

Procedural Programming

In a procedural approach, the system is divided into functions. Data is maintained using separate variables or global variables, and functions operate on that data.

Example:

```
double balance = 100.00;
double baseFare = 20.00;

double calculateFare(double fare, bool peak)
{
   if (peak)
      fare = fare + fare * 0.20;

   return fare;
}

bool canTravel(double balance, double fare)
{
   return (balance - fare) >= 50.00;
}
```

Here, the balance and fare are separate from the functions that process them. Any function that can access the balance may modify it, so the programmer must carefully maintain the minimum balance rule.

Object-Oriented Programming

In an object-oriented approach, related data and functions are combined into classes. Objects are created from these classes and interact with one another.

For example:

- The SmartCard class stores and manages the card balance.
- The Passenger class maintains passenger category information.

- The Journey class calculates the journey fare.
- The FareTable class maintains the fare values and surcharge rules.
- The Zone representation identifies the zones travelled.

This approach improves data protection, modularity and maintainability. Changes to one class can often be made without modifying unrelated parts of the program.

1.4 Basic Principles of Object-Oriented Programming

1. Encapsulation

Encapsulation combines data and related functions inside a class and restricts direct access to sensitive data.

Example: The SmartCard class keeps its balance private and provides controlled methods such as deductFare() and getBalance().

This prevents other parts of the program from directly assigning an invalid or negative balance.

2. Abstraction

Abstraction hides internal implementation details and exposes only the necessary operations.

Example: The Journey object can provide a calculateFare() method. The user of the method does not need to know every internal calculation involved in applying the base fare, peak surcharge and concession.This reduces complexity and makes the system easier to use.

3. Inheritance

Inheritance allows a new class to reuse the properties and functions of an existing class.

Example: Student and SeniorCitizen classes can inherit common passenger information from a Passenger base class.

Each derived class can provide its own concession-related behaviour. This avoids repeating common passenger information.

4. Polymorphism

Polymorphism allows the same function interface to perform different operations depending on the object involved.

Example: A virtual getConcession() function in the Passenger class can be overridden by Student and SeniorCitizen classes to calculate their respective concessions.

This makes it easier to introduce additional passenger categories without changing the entire fare-processing structure.

1.5 Rules Every Correct Journey Must Satisfy

The following rules must be maintained by the fare system:

No.	Rule	Explanation
1	Correct zone fare	The fare must be based on the farthest zone travelled: Inner, Middle or Outer.
2	Peak-hour surcharge	A 20% surcharge must be applied during 8:00–10:00 AM and 5:00–8:00 PM.
3	Off-peak fare	The normal base fare must be applied outside the specified peak periods.
4	Passenger concession	Eligible students and senior citizens must receive the applicable concession.
5	Minimum balance	At least Rs. 50 must remain on the card after the fare is deducted.
6	No negative balance	The card balance must never become negative.

7	Accurate calculation	Fare calculations must avoid rounding errors and produce the correct payable amount.
8	Consistent fare values	Fare values and limits should be maintained centrally to avoid inconsistent values throughout the program.

The minimum balance condition can be represented as:

Journey permitted when:

Remaining Balance = Current Balance − Fare

Condition:

Remaining Balance ≥ Rs. 50

If this condition is not satisfied, the journey must be denied and the card balance must remain unchanged.

1.6 Alternative Concept and Complexity Analysis – Procedural Design

If the system is implemented using a purely procedural approach, global variables can store fare values, surcharge percentage and minimum balance. Free functions can calculate fares, check peak hours and validate card balances.

Example procedural pseudocode:

```
START

INPUT cardBalance
INPUT zone
INPUT journeyTime

SELECT baseFare USING zone
```

```
IF journeyTime is within peak hours
    APPLY 20% surcharge
END IF

APPLY applicable passenger concession

IF cardBalance - fare >= 50
    cardBalance = cardBalance - fare
    DISPLAY fare and remaining balance
ELSE
    DISPLAY "Journey Denied"
END IF

STOP
```

Complexity Comparison

Factor	**Procedural Approach**	**Object-Oriented Approach**
Time per journey	O(1)	O(1)
Memory per card	O(1)	O(1)
Data organisation	Separate variables and functions	Data and behaviour grouped into classes
Data protection	Depends on programmer-controlled access	Private data and controlled methods
Maintainability	Changes may affect several functions	Changes can be isolated within relevant classes

Rule enforcement	Must be checked by every relevant function	Can be enforced through class methods

Time Complexity: Both approaches require a constant number of operations for one journey. Therefore, the time complexity is O(1).

Space Complexity: Both approaches store a fixed number of values for each card. Therefore, memory per card is O(1).

Impact of Changing a Fare Rule: In a procedural design, a fare change may require modifying multiple functions or global values if the rule is repeated in different places. In an object-oriented design, the change can generally be made in the relevant class, such as FareTable or Journey. This reduces the possibility of inconsistent updates.

1.7 Conclusion

The object-oriented approach is more suitable for the Namma Metro Smart-Card Fare System because it organises passengers, cards, journeys, zones and fare rules into separate classes. Encapsulation protects card balances, abstraction simplifies fare operations, inheritance supports passenger categories and polymorphism allows different concession behaviours.

Although procedural and object-oriented approaches have the same O(1) time complexity per journey and O(1) memory complexity per card, the object-oriented approach provides better data protection, modularity and maintainability. Therefore, an object-oriented design is recommended for developing and extending the metro fare system.